

Vertex UI Runtime Specification

Version: 1.0

Status: Draft

1. Objetivo

O UI Runtime é o subsistema do Vertex Core responsável por executar e renderizar as interfaces definidas pelo UI Engine.

Seu objetivo é transformar a árvore declarativa de componentes em uma interface gráfica interativa, eficiente e otimizada para dispositivos Wear OS.

Enquanto o UI Engine define o que deve ser exibido, o UI Runtime define como isso é executado e apresentado ao usuário.

2. Responsabilidades

O UI Runtime é responsável por:

- renderizar componentes;
- executar a composição da interface;
- aplicar atualizações visuais;
- processar entrada do usuário;
- executar animações;
- controlar foco;
- gerenciar o ciclo de vida das telas;
- otimizar desempenho gráfico;
- minimizar consumo de CPU, GPU, RAM e bateria.

3. Arquitetura

UI Engine

↓

UI Runtime

- ├── Renderer
- ├── Composition Runtime
- ├── Layout Runtime
- └── Input Runtime

- Animation Runtime
- Focus Manager
- Navigation Runtime
- Rendering Scheduler
- Frame Optimizer
- Runtime Metrics

↓

Jetpack Compose Runtime

↓

Android Graphics

↓

Display

O UI Runtime é a implementação concreta do modelo definido pelo UI Engine.

4. Componentes

Renderer

Renderiza os componentes na tela.

É responsável apenas pela apresentação visual.

Composition Runtime

Executa recomposições quando a interface sofre alterações.

Atualiza apenas os componentes afetados.

Layout Runtime

Calcula posições e tamanhos finais utilizados durante a renderização.

Input Runtime

Processa entradas provenientes do sistema.

Suporta:

- toque;
- gestos;
- botão físico;
- coroa rotativa;
- futuros dispositivos de entrada.

Animation Runtime

Executa animações.

Controla:

- interpolação;
- duração;
- sincronização;
- cancelamento.

Focus Manager

Gerencia foco da interface.

Especialmente importante para Wear OS.

Navigation Runtime

Executa transições entre telas.

Rendering Scheduler

Agenda renderizações.

Evita renderizações redundantes.

Frame Optimizer

Monitora desempenho.

Objetivos:

- reduzir recomposição;
- reduzir renderizações;
- manter fluidez.

Runtime Metrics

Coleta métricas da interface.

5. Fluxos

Renderização Inicial

UI Engine

↓

Composition

↓

Layout

↓

Renderer

↓

Display

Atualização

State Change

↓

Composition Runtime



Layout Runtime



Renderer



Novo Frame

Entrada

Usuário



Input Runtime



Componente



Ação

Navegação

Screen A



Navigation Runtime



Animation



Screen B

6. APIs

Renderização

- render()
- invalidate()

Layout

- relayout()

Entrada

- dispatchInput()

Navegação

- showScreen()
- closeScreen()

Animação

- startAnimation()
- stopAnimation()

Diagnóstico

- renderingMetrics()
- currentFrameTime()

7. Estados

Uma tela pode assumir:

Created



Composing



Layout



Rendering



Visible



Paused



Hidden



Destroyed

8. Políticas

Renderização Sob Demanda

A interface só deve ser renderizada quando necessário.

Atualização Parcial

Somente componentes alterados devem ser recompostos.

Economia de Energia

Evitar renderizações contínuas sem necessidade.

Fluidez

O Runtime deve buscar manter animações suaves respeitando os limites do dispositivo.

Responsividade

Entradas do usuário devem possuir baixa latência.

Independência

O UI Runtime permanece desacoplado do modelo lógico do UI Engine.

9. Integrações

O UI Runtime integra-se com:

- UI Engine;
- Plugin API;
- Plugin Context;
- State Engine;
- Event System;
- Scheduler;
- Resource Manager;
- Memory Management;
- Power Management;
- Asset Manager;
- Android Bridge;
- Configuration System;
- Logging Framework.

Sua implementação padrão utiliza:

- Jetpack Compose Runtime;
- Android View System (quando necessário);
- APIs gráficas do Android.

10. Métricas

O UI Runtime registra:

- tempo de renderização;
- tempo de composição;
- tempo de layout;
- tempo médio por frame;
- quadros perdidos (dropped frames);
- recomposições;
- animações ativas;
- latência de entrada;
- consumo de memória gráfica;
- uso de CPU durante renderização.

Essas métricas são utilizadas pelo Plugin Inspector e pelo Logging Framework.

11. Exemplos

Exibição de uma Tela

1. O UI Engine fornece a árvore de componentes.
2. O UI Runtime executa a composição.
3. O layout é calculado.
4. O Renderer desenha a interface.
5. A tela é apresentada ao usuário.

Mudança de Estado

1. O "battery.level" é atualizado.
2. O componente correspondente é invalidado.
3. Apenas esse componente é recomposto.
4. Um novo frame é renderizado.

Rolagem pela Coroa Rotativa

1. O usuário gira a coroa.
2. O Input Runtime recebe o evento.
3. O componente focado processa a rolagem.
4. O Renderer atualiza apenas a região modificada.

Navegação

1. O usuário abre uma nova tela.
2. O Navigation Runtime inicia a transição.
3. A animação é executada.
4. A tela anterior é descartada quando apropriado.

12. Regras Arquiteturais

- O UI Runtime nunca define a estrutura da interface.
- Toda estrutura pertence ao UI Engine.
- A renderização deve ser incremental sempre que possível.
- Componentes invisíveis não devem consumir recursos desnecessários.
- O Runtime deve cooperar com o Power Management para reduzir consumo energético.
- A renderização deve respeitar os orçamentos definidos pelo Resource Manager.

13. Limitações

O UI Runtime não é responsável por:

- definir componentes;
- armazenar estados da aplicação;
- implementar regras de negócio;
- gerenciar plugins;
- controlar permissões.

Essas responsabilidades pertencem aos respectivos subsistemas.

14. Futuras Extensões

O UI Runtime poderá evoluir para suportar:

- renderização adaptativa baseada na bateria;
- redução dinâmica da taxa de atualização em telas estáticas;
- pré-renderização de telas frequentes;
- otimizações específicas para diferentes fabricantes de relógios;
- aceleração gráfica adicional quando disponível;
- perfis automáticos de desempenho.

15. Resumo

O UI Runtime é o mecanismo de execução da interface do Vertex. Ele transforma a descrição declarativa produzida pelo UI Engine em uma interface gráfica responsiva e eficiente, gerenciando renderização, entrada do usuário, animações e ciclo de vida visual. Sua arquitetura foi projetada para aproveitar os recursos do Jetpack Compose e do Android, mantendo baixo consumo de CPU, memória e bateria, requisitos essenciais para dispositivos Wear OS.